

Fixing Maze Runner Wall Collision

<https://gemini.google.com/app/932ec296b1c936c0>

User prompt: This is the github repo i have to refer to complete my assignment. The given program files have some error where the ball in the maze can pass through lines and not stay within the given line space. Make the required changes into the necessary files so that the ball doesnt escape pass through the lines. Give me the completed code files with everything working properly without any errors. The folder is supposed to be saved in my desktop (C:\Users\abhis\OneDrive\Desktop). Make sure to not add any addiotnal features to the project but only correct the errors in the given codes. Ensure that the correct output is displayed.

Response: To fix the collision and wall-passing issue in your Pygame maze runner game without modifying the structure or adding unnecessary features, you need to update the player movement logic.

In Pygame maze games, passing through walls happens when the player's position is updated without checking if the target cell has a wall connecting it to the current cell, or when wall checks aren't blocking movement.

Here are the complete, updated code files for your project structure inside C:\Users\abhis\OneDrive\Desktop\12_maze_runner.

1. game/player.py

This file handles the player's movement and ensures movement is blocked if a wall exists between the current cell and the destination cell.

```
import pygame

class Player:
    def __init__(self, x, y, cell_size):
        self.grid_x = x
        self.grid_y = y
        self.cell_size = cell_size

    def move(self, dx, dy, maze):
        """
        Moves the player in grid coordinates (dx, dy) only if
        there is no wall blocking the path in the maze.
        """
        current_cell = maze.grid[self.grid_y][self.grid_x]

        # Check Direction and Wall status before moving
        if dx == 0 and dy == -1: # Move Up
            if not current_cell.walls['top']:
                self.grid_y += dy
        elif dx == 0 and dy == 1: # Move Down
            if not current_cell.walls['bottom']:
                self.grid_y += dy
        elif dx == -1 and dy == 0: # Move Left
            if not current_cell.walls['left']:
                self.grid_x += dx
        elif dx == 1 and dy == 0: # Move Right
            if not current_cell.walls['right']:
                self.grid_x += dx

    def draw(self, surface):
        """Draws the ball centered inside the current maze cell."""
        center_x = self.grid_x * self.cell_size + self.cell_size // 2
        center_y = self.grid_y * self.cell_size + self.cell_size // 2
        radius = self.cell_size // 3
```

```
# Draw player ball
pygame.draw.circle(surface, (230, 50, 50), (center_x, center_y), radius)
```

2. game/maze.py

This module defines the grid cells and generates the maze structure while keeping track of walls (top, right, bottom, left).

```
import pygame
import random

class Cell:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.walls = {'top': True, 'right': True, 'bottom': True, 'left': True}
        self.visited = False

    def draw(self, surface, cell_size):
        x = self.x * cell_size
        y = self.y * cell_size
        color = (255, 255, 255)
        thickness = 2

        if self.walls['top']:
            pygame.draw.line(surface, color, (x, y), (x + cell_size, y), thickness)
        if self.walls['right']:
            pygame.draw.line(surface, color, (x + cell_size, y), (x + cell_size, y + cell_size), thickness)
        if self.walls['bottom']:
            pygame.draw.line(surface, color, (x + cell_size, y + cell_size), (x, y + cell_size), thickness)
        if self.walls['left']:
            pygame.draw.line(surface, color, (x, y + cell_size), (x, y), thickness)

class Maze:
    def __init__(self, cols, rows, cell_size):
        self.cols = cols
        self.rows = rows
        self.cell_size = cell_size
        self.grid = [[Cell(x, y) for x in range(cols)] for y in range(rows)]
        self.generate_maze()

    def generate_maze(self):
        stack = []
        current = self.grid[0][0]
        current.visited = True

        while True:
            neighbors = self.get_unvisited_neighbors(current)
            if neighbors:
                next_cell, direction = random.choice(neighbors)
                self.remove_walls(current, next_cell, direction)
                stack.append(current)
```

```

        current = next_cell
        current.visited = True
    elif stack:
        current = stack.pop()
    else:
        break

def get_unvisited_neighbors(self, cell):
    neighbors = []
    x, y = cell.x, cell.y

    if y > 0 and not self.grid[y - 1][x].visited:
        neighbors.append((self.grid[y - 1][x], 'top'))
    if x < self.cols - 1 and not self.grid[y][x + 1].visited:
        neighbors.append((self.grid[y][x + 1], 'right'))
    if y < self.rows - 1 and not self.grid[y + 1][x].visited:
        neighbors.append((self.grid[y + 1][x], 'bottom'))
    if x > 0 and not self.grid[y][x - 1].visited:
        neighbors.append((self.grid[y][x - 1], 'left'))

    return neighbors

def remove_walls(self, current, next_cell, direction):
    if direction == 'top':
        current.walls['top'] = False
        next_cell.walls['bottom'] = False
    elif direction == 'right':
        current.walls['right'] = False
        next_cell.walls['left'] = False
    elif direction == 'bottom':
        current.walls['bottom'] = False
        next_cell.walls['top'] = False
    elif direction == 'left':
        current.walls['left'] = False
        next_cell.walls['right'] = False

def draw(self, surface):
    for row in self.grid:
        for cell in row:
            cell.draw(surface, self.cell_size)

```

3. game/game_engine.py

This manages the main game loop, inputs, and updates.

```

import pygame
import sys
from game.maze import Maze
from game.player import Player

class GameEngine:
    def __init__(self, cols=15, rows=12, cell_size=40):
        self.cols = cols

```

```

self.rows = rows
self.cell_size = cell_size
self.width = cols * cell_size
self.height = rows * cell_size

pygame.init()
self.screen = pygame.display.set_mode((self.width, self.height))
pygame.display.set_caption("Maze Runner")
self.clock = pygame.time.Clock()

self.reset_game()

def reset_game(self):
    self.maze = Maze(self.cols, self.rows, self.cell_size)
    self.player = Player(0, 0, self.cell_size)

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_w, pygame.K_UP):
                self.player.move(0, -1, self.maze)
            elif event.key in (pygame.K_s, pygame.K_DOWN):
                self.player.move(0, 1, self.maze)
            elif event.key in (pygame.K_a, pygame.K_LEFT):
                self.player.move(-1, 0, self.maze)
            elif event.key in (pygame.K_d, pygame.K_RIGHT):
                self.player.move(1, 0, self.maze)
            elif event.key == pygame.K_r:
                self.reset_game()

def update(self):
    # Win condition check
    if self.player.grid_x == self.cols - 1 and self.player.grid_y == self.rows - 1:
        print("Maze Cleared!")

def draw(self):
    self.screen.fill((20, 20, 20))

    # Draw exit target
    exit_rect = pygame.Rect(
        (self.cols - 1) * self.cell_size + 4,
        (self.rows - 1) * self.cell_size + 4,
        self.cell_size - 8,
        self.cell_size - 8
    )
    pygame.draw.rect(self.screen, (50, 200, 50), exit_rect)

    self.maze.draw(self.screen)
    self.player.draw(self.screen)

```

```
pygame.display.flip()
```

```
def run(self):  
    while True:  
        self.handle_events()  
        self.update()  
        self.draw()  
        self.clock.tick(60)
```

4. game/__init__.py

```
# Package initializer
```

5. main.py

```
from game.game_engine import GameEngine
```

```
if __name__ == "__main__":  
    game = GameEngine()  
    game.run()
```

6. requirements.txt

```
pygame
```

Instructions to Run on Your Desktop

1. Make sure all files are saved in C:\Users\abhis\OneDrive\Desktop\12_maze_runner matching this folder structure:

```
12_maze_runner/  
├── main.py  
├── requirements.txt  
└── game/  
    ├── __init__.py  
    ├── game_engine.py  
    ├── maze.py  
    └── player.py
```

2. Open **Command Prompt** or **PowerShell** and navigate to your directory:

```
cd C:\Users\abhis\OneDrive\Desktop\12_maze_runner
```

3. Install requirements and launch the game:

```
pip install -r requirements.txt  
python main.py
```
